University of Science and Technology of Oran – Mohamed Boudiaf
Faculty of Mathematics and Informatics / **Department of Computer Science**
Master's Degree Program – Specializations: **Artificial Intelligence**, **Data Science**, **Networks and Distributed Systems**
Subject : Advanced Algorithms and Complexity [2026 – 2027]

Tutorial Sheet — TD 3

Data Structures and Complexity

Corresponds to: Lecture 4

Learning Objectives

- Compare the time complexity of core operations across fundamental data structures.
- Apply the aggregate method of amortised analysis to a dynamic array.
- Manipulate a binary heap represented as an array.

Exercise 1 — Solved in class

Complete the following table with the worst-case time complexity of each operation for each data structure.

Structure	Search	Insert	Delete	Find-Min
Unsorted array (n elts)				
Sorted array (n elts)				
Unbalanced BST				
Balanced BST (e.g. AVL)				
Binary min-heap				
Hash table (chaining, avg.)				

Exercise 2 — Solved in class

A dynamic array starts empty and doubles its capacity (copying all existing elements) whenever it becomes full. Using the aggregate method, show that a sequence of **n** insertions costs **O(n)** in total, i.e. **O(1)** amortised per insertion.

Exercise 3 — For the student (model answer provided below)

Insert the values **5, 3, 8, 1, 9, 2** one at a time into an initially empty binary min-heap (using the standard array representation and "bubble-up" after each insertion).

1. Show the array representation of the heap after each of the six insertions.
2. Explain, in one or two sentences, why a min-heap does not support searching for an arbitrary value in **O(log n)** time, even though it supports insertion and find-min in **O(log n) / O(1)**.